

A PROJECT ON HOUSEHOLD SERVICE MANAGEMENT

**SUBMITTED IN
PARTIAL FULFILLMENT OF THE REQUIREMENT
FOR THE COURSE OF DIPLOMA IN ADVANCED COMPUTING
FROM CDAC**



Hinjawadi, Pune

SUBMITTED BY
Rupprashik Anand Khare

UNDER THE GUIDANCE OF
Prof. Pooja Jaiswal
Faculty Member
Sunbeam Institute of Information Technology, Pune

ACKNOWLEDGEMENT

A project usually falls short of its expectation unless aided and guided by the right persons at the right time. We avail this opportunity to express our deep sense of gratitude towards Mr. Nitin Kudale (Center Coordinator, SIIT, Pune) and Mr. Yogesh Kolhe (Course Coordinator, SIIT, Pune).

We are deeply indebted and grateful to them for their guidance, encouragement, and deep concern for our project. Without their critical evaluation and suggestions at every stage of the project, this project could never have reached its present form.

Last but not the least, we thank the entire faculty and staff members of Sunbeam Institute of Information Technology, Pune for their support.

Rupprashik Anand Khare
0225 PG-DAC
SIIT Pune

Academic Year: 2025–2026

Contents

1	Introduction to Project	3
2	Requirements	4
2.1	Functional Requirements	4
2.1.1	System Overview	5
2.1.2	Registration and Creation of User Profile	5
2.1.3	Actor 1 – Customer	5
2.1.4	Actor 2 – Service Provider	7
2.1.5	Actor 3 – Admin	7
2.1.6	Main Functionalities Summary	8
2.2	Non-Functional Requirements	8
2.2.1	Interface	8
2.2.2	Performance	9
2.2.3	Constraints	9
2.2.4	Other Requirements	9
3	Design	10
3.1	Database Design	10
3.1.1	Users Table	10
3.1.2	Service Providers Table	10
3.1.3	Service Provider Documents Table	11
3.1.4	Service Categories Table	11
3.1.5	Services Table	11
3.1.6	Provider Skills Table	11
3.1.7	Bookings Table	12
3.1.8	Payments Table	12
3.1.9	Reviews Table	12
3.1.10	Disputes Table	12
3.1.11	System Logs Table	13
4	Coding Standards Implemented	14
4.1	Naming and Capitalization	14
4.2	Comments	14
5	Test Report	15
6	Project Management Related Statistics	16
	Appendix A	17
	References	30

1 Introduction to Project

With the rapid growth of digital technologies and internet accessibility, service-based platforms have transformed the way customers access professional services. Traditional methods of finding and booking services are often time-consuming, inefficient, and lack transparency. To address these challenges, a centralized and secure digital solution is required that connects customers with verified service providers while ensuring smooth booking, payment, and management processes.

The Service Booking and Management System is a web-based application designed to facilitate seamless interaction between customers, service providers, and administrators. The system allows customers to browse available services, schedule bookings, make online payments, track booking status, and raise disputes when required. Service providers can manage incoming service requests, update booking status, and monitor their service performance. Administrators oversee the entire platform, ensuring proper system functioning, user management, and dispute resolution.

The application is developed using Spring Boot for backend services and React.js for the frontend, following a modular and scalable architecture. Secure authentication and authorization are implemented using JWT (JSON Web Token) to protect user data and enable role-based access control. Online payment functionality is integrated using Razorpay, allowing customers to perform secure and reliable transactions within the system.

The system is designed with a focus on usability, security, and performance. By digitizing the service booking workflow, the project aims to reduce manual effort, improve transparency, and enhance the overall user experience. The proposed solution provides a robust foundation for scalable service-oriented platforms and demonstrates the practical implementation of modern web technologies in solving real-world problems.

2 Requirements

2.1 Functional Requirements

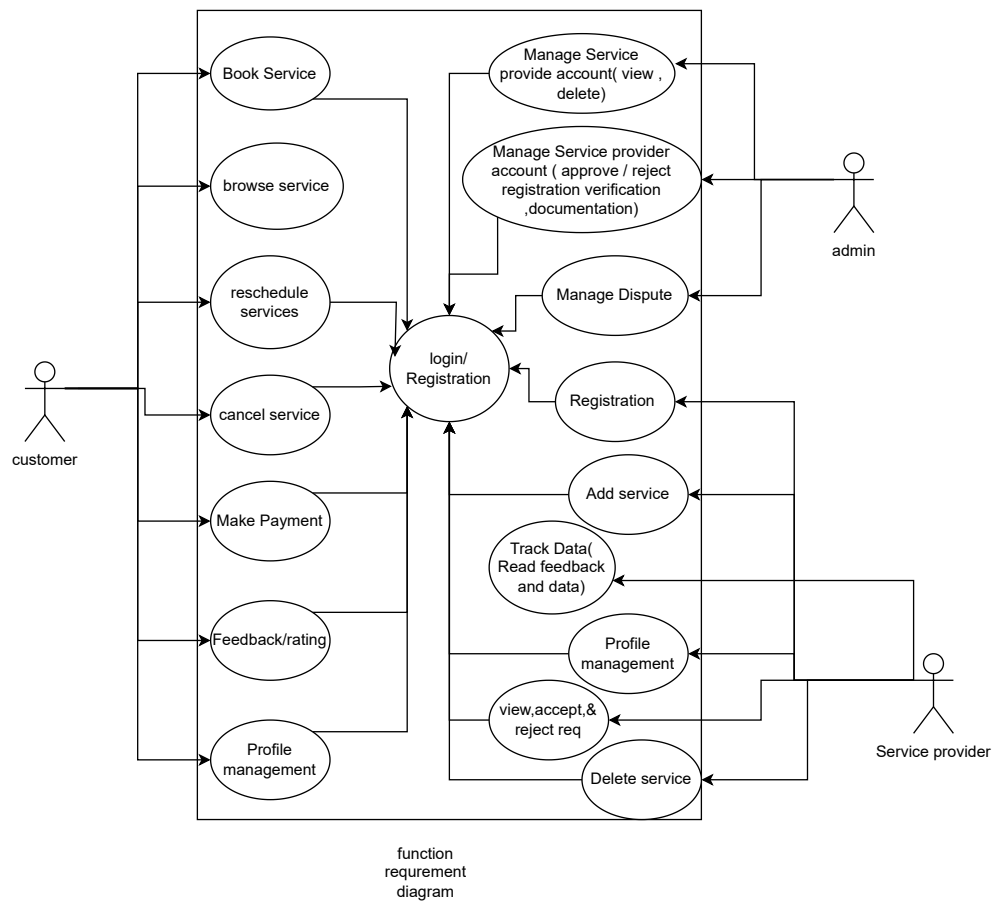


Figure 1: Functional Requirement Diagram

2.1.1 System Overview

The system follows a multi-actor architecture consisting of:

- Customer
- Service Provider
- Admin

The system follows a multi-actor architecture consisting of Customer, Service Provider, and Admin. Each actor interacts with the system through secure, role-based APIs using JWT authentication. The Use Case Diagram represents the interactions between actors and system functionalities.

Use Case Diagram is recommended here.

2.1.2 Registration and Creation of User Profile

Users can:

- Register using email and password
- Login using JWT authentication
- Create and update their profile
- Upload profile images

Activity Diagram or Sequence Diagram is recommended here.

2.1.3 Actor 1 – Customer

Customer functionalities include:

- **User registration and login** The customer can register on the platform by providing valid email and password details. During login, the system verifies the credentials and generates a JWT token. This ensures secure and authenticated access to customer-specific features.
- **Profile management** Customers can create and manage their personal profile within the system. This includes updating contact information, address, and other relevant details. Profile data is securely stored and can be modified at any time.
- **Viewing service categories** The system allows customers to browse various service categories available on the platform. Each category displays a list of services along with descriptions and pricing details. This helps customers make informed service selections.

- **Booking services** Customers can book required services by selecting the service type, preferred date, and time slot. Once a booking request is placed, it is forwarded to the respective service provider. Booking details are stored for future reference.
- **Cancelling bookings** Customers are provided with the option to cancel bookings before service execution. The system updates the booking status accordingly and notifies the service provider. This ensures flexibility and better user experience.
- **Online payment using Razorpay** The system integrates Razorpay to handle on-line payments securely. Customers can complete payments using multiple payment options supported by Razorpay. Transaction details are recorded for verification and tracking.
- **Viewing payment history** Customers can view a detailed history of all completed and pending payments. This includes transaction dates, amounts, and payment status. Payment history helps customers maintain transparency and records.
- **Raising disputes** In case of any service-related issues, customers can raise disputes through the platform. The dispute details are forwarded to the admin for review and resolution. This feature ensures accountability and trust.
- **Submitting reviews** After successful service completion, customers can submit reviews and ratings for the service provider. Reviews help improve service quality and guide other customers in decision-making.

2.1.4 Actor 2 – Service Provider

Service Provider functionalities include:

- **Login and profile management** Service providers can log in to the system using secure credentials with JWT authentication. They can create and update their profile, including service details, availability, and contact information. This ensures accurate representation on the platform.
- **Viewing assigned bookings** Service providers can view all bookings assigned to them through their dashboard. Booking details include customer information, service type, date, and time. This helps providers plan and manage their work efficiently.
- **Accepting or rejecting bookings** After reviewing booking requests, service providers can accept or reject them based on availability. The system updates the booking status accordingly and notifies the customer. This ensures proper communication and scheduling.
- **Updating booking status** Service providers can update the status of bookings as pending, in-progress, or completed. Status updates help customers track service progress in real time. This improves transparency and service reliability.
- **Viewing payment history** Service providers can view payment details for completed services. The payment history includes transaction amounts, dates, and payment status. This helps in monitoring earnings and financial records.
- **Managing offered services** Service providers can add, update, or remove services they offer on the platform. Service details such as pricing, description, and availability can be modified. This allows flexibility in service management.
- **Viewing dashboard statistics** The system provides a dashboard displaying key performance statistics such as total bookings, completed services, and earnings. These insights help service providers analyze performance and improve service delivery.

2.1.5 Actor 3 – Admin

Admin functionalities include:

- **User management** The admin is responsible for managing all users on the platform, including customers and service providers. This includes viewing user details, activating or deactivating accounts, and maintaining system integrity.
- **Monitoring platform activities** The admin continuously monitors platform operations such as bookings, payments, and service activities. This helps ensure smooth system functioning and quick identification of irregular activities.

- **Resolving disputes** Admins handle disputes raised by customers or service providers related to services or payments. After reviewing dispute details, the admin takes appropriate actions to ensure fair resolution and transparency.
- **Viewing system-wide statistics** The system provides the admin with access to overall platform statistics such as total users, bookings, revenue, and service performance. These statistics support decision-making and system evaluation.
- **Managing categories and services** The admin can create, update, or remove service categories and services available on the platform. This ensures that the system remains organized and up-to-date with user requirements.

2.1.6 Main Functionalities Summary

- **Secure authentication using JWT** The system uses JWT (JSON Web Token) authentication to verify user identity. Tokens are generated during login and attached to each request, ensuring secure and stateless communication.
- **Role-based access control** Access to system features is controlled based on user roles such as Customer, Service Provider, and Admin. This prevents unauthorized access and ensures data security.
- **Service booking and management** The platform allows customers to book services and service providers to manage bookings efficiently. Booking statuses are updated in real time to maintain transparency.
- **Online payment integration** Secure online payments are supported through Razorpay integration. The system records all transactions and ensures safe payment processing.
- **Dispute and review handling** Users can raise disputes related to services or payments, which are handled by the admin. Customers can also submit reviews and ratings to improve service quality.

2.2 Non-Functional Requirements

2.2.1 Interface

- Responsive web interface
- User-friendly dashboards
- Consistent UI using Tailwind CSS

Refer Appendix B for detailed UI screenshots.

2.2.2 Performance

Number of Concurrent Users: The system supports multiple concurrent users using stateless JWT authentication.

Booking of Services: Booking operations are optimized using indexed database queries.

2.2.3 Constraints

- Requires stable internet connection
- Depends on Razorpay availability for payments
- JWT token expiry limits session duration

2.2.4 Other Requirements

Hardware Interfaces

- Minimum 4GB RAM
- Internet-enabled device (PC / Laptop / Mobile)

Software Interfaces

- Web Browser (Chrome, Firefox)
- Backend: Java 17+, Spring Boot
- Frontend: React.js
- Database: MySQL

3 Design

3.1 Database Design

The system uses a relational database to manage users, services, bookings, payments, and related activities. The database is normalized to avoid redundancy and maintain data integrity. Detailed ER Diagram, Data Flow Diagram, and Class Diagram are provided in Appendix A.

3.1.1 Users Table

Column Name	Description
user_id	Unique identifier for each user (Primary Key)
first_name, last_name	Name of the user
email	Registered email address
password	Encrypted login password
phone	Contact number
gender	Gender of the user
dob	Date of birth
role	User role (Customer / Service Provider / Admin)
street, city, state, pin-code	Address details
profile_image	Profile picture URL
preferences	User preferences
is_active	Account active status
deactive_reason	Reason for account deactivation
deactive_time	Account deactivation time
created_on, up-dated_on	Record timestamps
last_login	Last login time

3.1.2 Service Providers Table

Column Name	Description
service_provider_id	Unique provider ID (Primary Key)
user_id	Linked user ID (FK → Users.user_id)
verification_status	Verification state
certification	Professional certification details
gov_id	Government ID number
gov_id_type	Type of government ID
created_on, up-dated_on	Record timestamps

3.1.3 Service Provider Documents Table

id	Document ID (Primary Key)
service_provider_id	FK → Service_Providers
document_type	Type of document
document_url	Uploaded document URL
public_id	Cloud storage identifier
verification_status	Approval status
rejection_reason	Reason for rejection
created_on, up- dated_on	Record timestamps

3.1.4 Service Categories Table

category_id	Category ID (Primary Key)
name	Category name
description	Category description
service_image	Category image
created_on, up- dated_on	Record timestamps

3.1.5 Services Table

service_id	Service ID (Primary Key)
name	Service name
description	Service description
category_id	FK → Service_Categories
base_price	Service base price
duration_minutes	Duration in minutes
is_available	Availability status
image_url	Service image URL
created_on, up- dated_on	Record timestamps

3.1.6 Provider Skills Table

service_provider_id	FK → Service_Providers
service_id	FK → Services

3.1.7 Bookings Table

booking_id	Booking ID (Primary Key)
user_id	FK → Users
provider_id	FK → Service_Providers
service_id	FK → Services
scheduled_at	Scheduled date and time
status	Booking status
price	Total service price
rejection_reason	Reason for rejection
created_on, up- dated_on	Record timestamps

3.1.8 Payments Table

payment_id	Payment ID (Primary Key)
booking_id	FK → Bookings
transaction_id	Payment gateway transaction ID
amount	Paid amount
method	Payment method
status	Payment status
created_on, up- dated_on	Record timestamps

3.1.9 Reviews Table

review_id	Review ID (Primary Key)
booking_id	FK → Bookings
user_id	FK → Users
rating	Rating value
comment	Review comment
created_on, up- dated_on	Record timestamps

3.1.10 Disputes Table

dispute_id	Dispute ID (Primary Key)
booking_id	FK → Bookings
raised_by_user_id	FK → Users
resolved_by_user_id	FK → Users
description	Dispute details
status	Dispute status
created_on, up- dated_on	Record timestamps

3.1.11 System Logs Table

log_id	Log ID (Primary Key)
actor_id	FK → Users
entity_name	Affected entity name
entity_id	Affected entity ID
action	Action performed
created_on, up- dated_on	Record timestamps

Each table is normalized to avoid redundancy.

Refer Appendix A for ER Diagram, DFD, and Class Diagram.

4 Coding Standards Implemented

4.1 Naming and Capitalization

Proper naming conventions and capitalization standards are followed throughout the project to ensure code readability and maintainability. Variables and methods are named using the CamelCase convention, which improves clarity and consistency in the codebase. Class names follow the PascalCase convention, making them easily distinguishable from variables and methods.

Meaningful and descriptive names are used for variables, methods, and classes so that the purpose of each component is clearly understood. This approach reduces ambiguity and helps developers quickly understand the program logic, making the system easier to debug and extend.

4.2 Comments

Comments are used effectively to explain the logic and functionality of the code. Inline comments are added where necessary to clarify complex logic and critical operations. These comments help in understanding the flow of execution and decision-making within the code.

Method-level comments are written to describe the purpose, input parameters, and expected output of each method. Additionally, API documentation is maintained using annotations, which helps in generating clear and structured API references. Proper commenting improves code quality, enhances collaboration, and simplifies future maintenance.

5 Test Report

The testing phase was conducted to validate the correctness, reliability, and performance of the system. The following table summarizes the test cases executed for major features and functionalities of the system.

Test Case ID	Feature / Module	Input	Expected Output	Actual Output	Status
TC01	User Registration	Valid user details	User account created successfully	User account created successfully	Pass
TC02	User Login	Valid email and password	Login successful and JWT generated	Login successful and JWT generated	Pass
TC03	Role-Based Access	Customer login	Access only customer dashboard	Customer dashboard displayed	Pass
TC04	View Service Categories	Customer request	List of service categories displayed	Service categories displayed	Pass
TC05	Service Booking	Valid booking details	Booking created successfully	Booking created successfully	Pass
TC06	Booking Acceptance	Provider accepts booking	Booking status updated to accepted	Booking status updated	Pass
TC07	Booking Cancellation	Cancel request before service	Booking cancelled successfully	Booking cancelled successfully	Pass
TC08	Online Payment	Razorpay test payment	Payment completed successfully	Payment completed successfully	Pass
TC09	Payment History	Customer request	Display past transactions	Transaction history displayed	Pass
TC10	Dispute Creation	Valid dispute details	Dispute recorded successfully	Dispute recorded successfully	Pass
TC11	Dispute Resolution	Admin resolves dispute	Dispute status updated	Dispute resolved	Pass
TC12	Review Submission	Rating and comment	Review saved successfully	Review saved successfully	Pass
TC13	Profile Update	Updated profile data	Profile updated successfully	Profile updated successfully	Pass
TC14	Dashboard Analytics	Admin request	System statistics displayed	Statistics displayed	Pass
TC15	Logout	Logout request	User session terminated	User logged out	Pass

6 Project Management Related Statistics

The QuickServe project was developed using the Agile SDLC approach. GitHub was used for version control to track development progress across different phases. The table below summarizes the SDLC stages, duration, and work carried out.

Phase	SDLC Stage	Duration	Work Done
1	Requirement Analysis	01 Sept 2025 – 30 Sept 2025	Analyzed problem statement, studied existing service platforms, identified user roles, and finalized functional and non-functional requirements based on initial discussions and planning.
2	System Design & Database Design	01 Oct 2025 – 31 Oct 2025	Designed overall system architecture, finalized database schema, entity relationships, and prepared ER diagram, DFD, and class diagram.
3	Frontend Development	01 Nov 2025 – 31 Dec 2025	Developed React.js frontend including homepage, authentication pages, customer dashboard, service provider dashboard, admin pages, navigation components, and UI enhancements.
4	Backend Development (REST APIs)	01 Jan 2026 – 24 Jan 2026	Implemented Spring Boot backend structure, REST APIs for users, services, bookings, reviews, disputes, and integrated JWT-based authentication and role-based access control.
5	Payment Integration	20 Jan 2026 – 31 Jan 2026	Integrated Razorpay payment gateway, implemented order creation and verification APIs, handled payment status, and linked transactions with bookings.
6	Testing	25 Jan 2026 – 02 Feb 2026	Performed unit testing and integration testing, fixed authentication issues, resolved API errors, improved security, and validated complete booking and payment workflows.
7	Deployment & Documentation	01 Feb 2026 – 03 Feb 2026	Finalized application configuration, resolved merge conflicts, updated GitHub repository, prepared documentation, screenshots, and project submission artifacts.

Appendix

- Entity Relationship Diagram

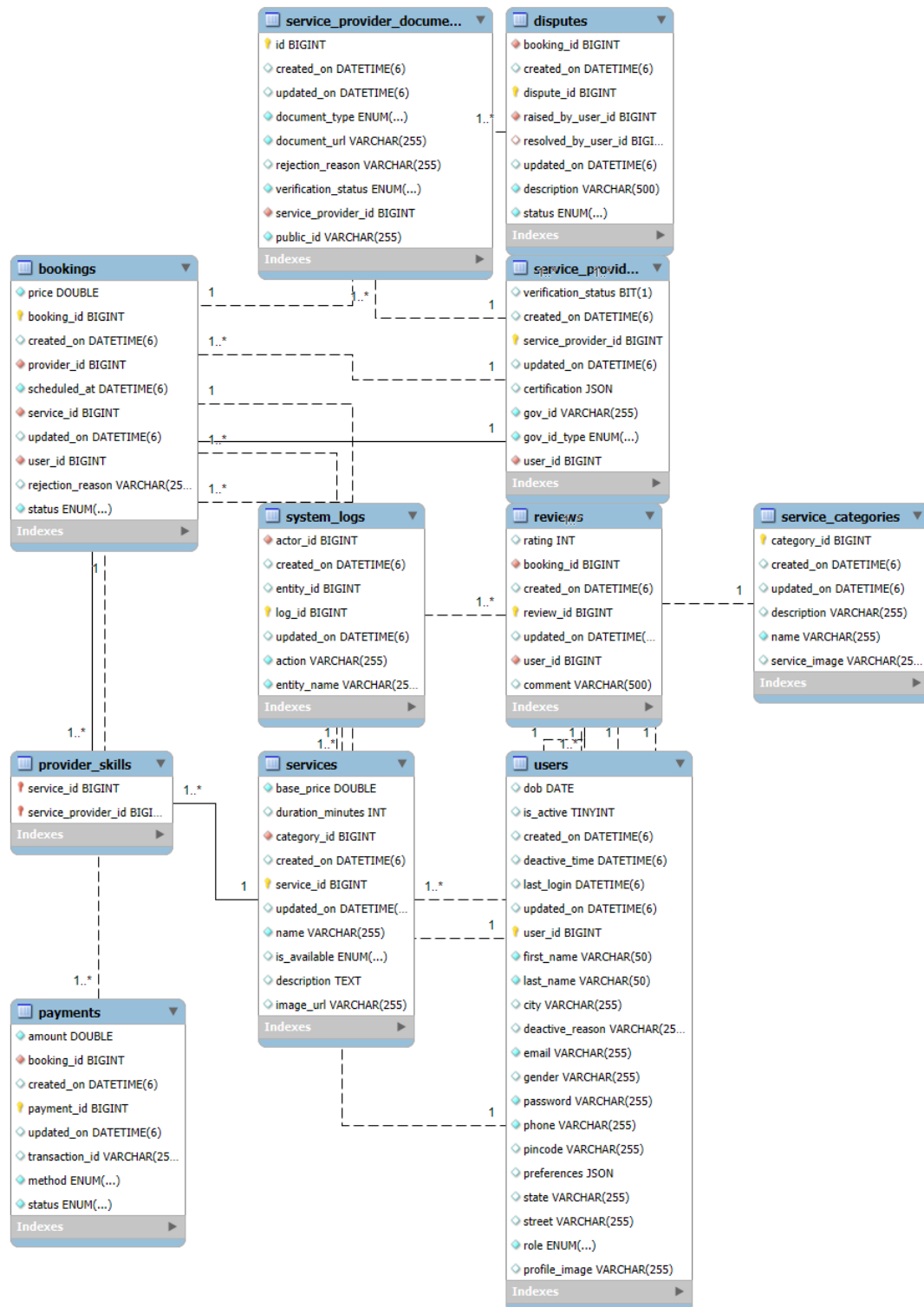


Figure 2: ER Diagram

- Data Flow Diagram

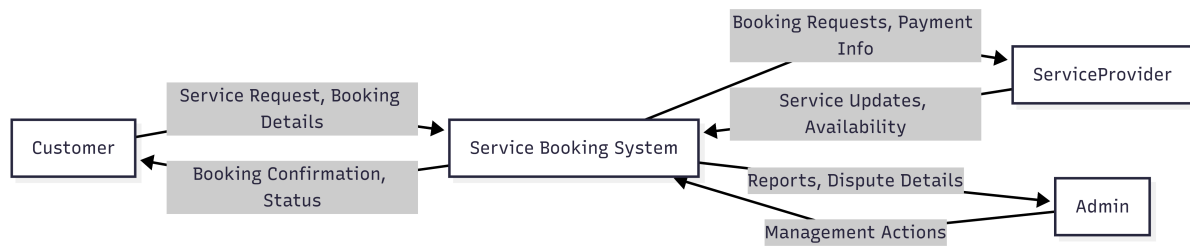


Figure 3: Data flow level-0 diagram

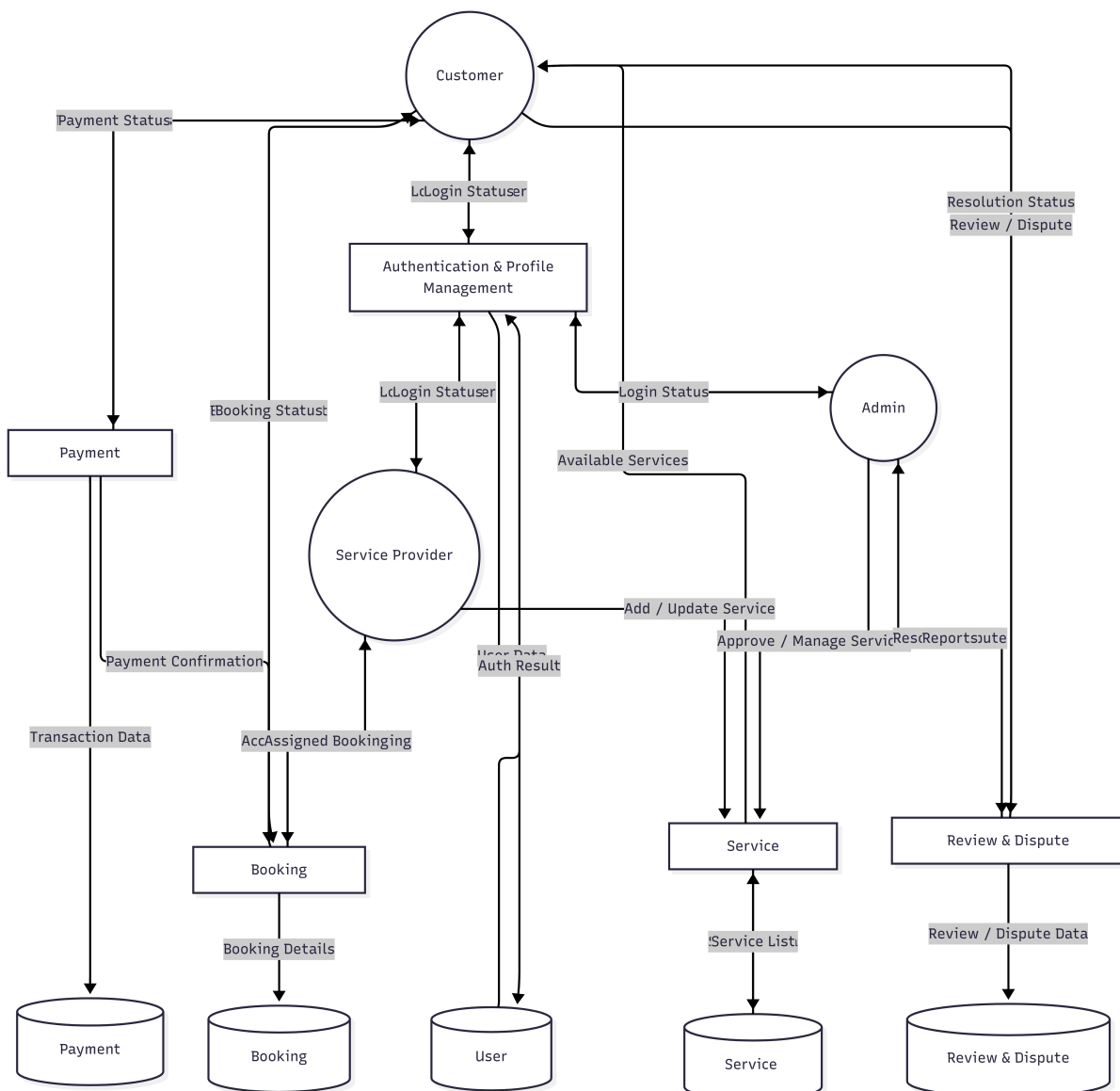


Figure 4: Data flow level-1 diagram

- Class Diagram

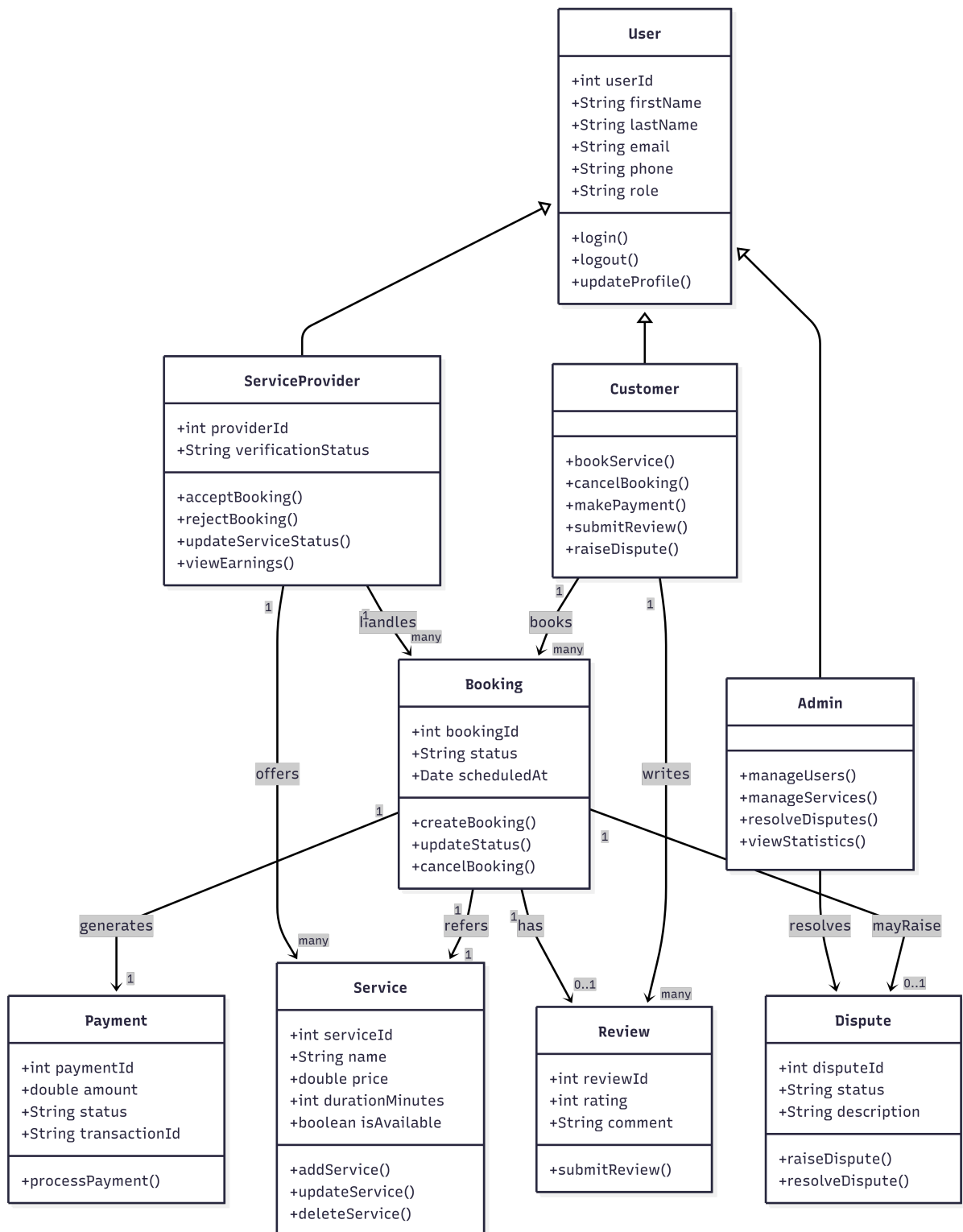


Figure 5: Class diagram

Homepage

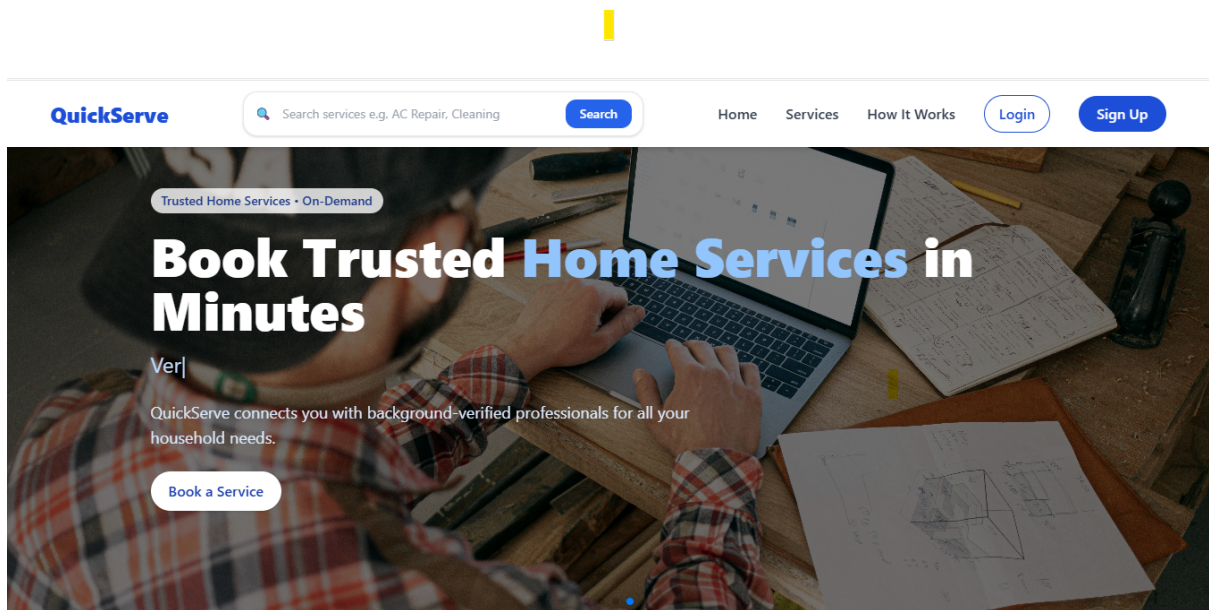


Figure 6: Homepage

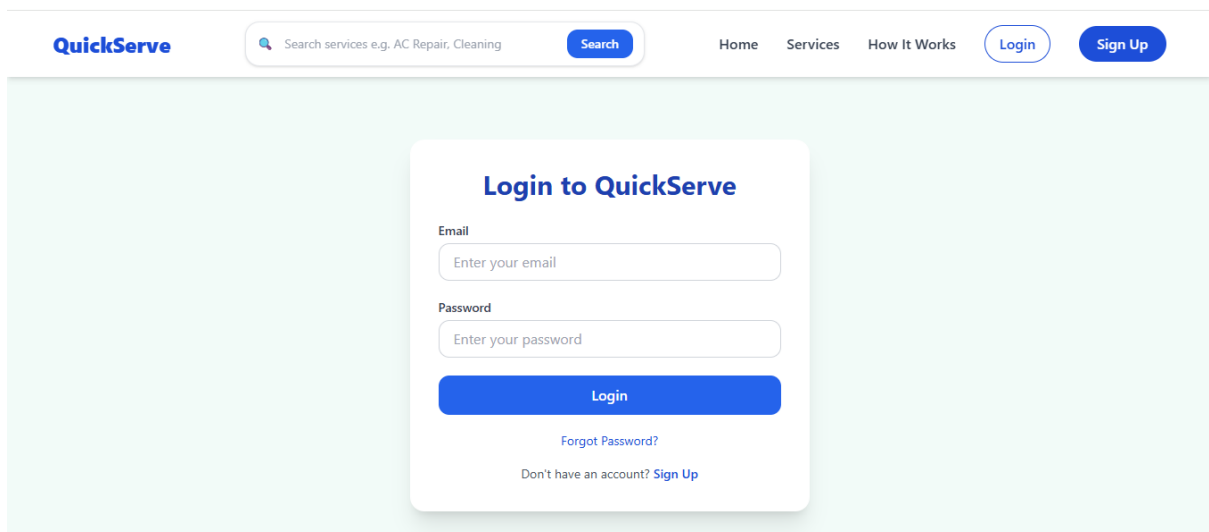


Figure 7: Login

QuickServe

Search services e.g. AC Repair, Cleaning

Search

Home

Services

How It Works

Login

Sign Up

Account

First Name

yashraj

Last Name

desale

Email

provider@gmail.com

Phone

8888138820

Password

Sign Up As

Service Provider

Sign Up

Figure 8: Register

Customer Dashboard

QuickServe
Customer Panel

Dashboard

Bookings

Payments

Profile

Support

Settings

Logout

Customer Dashboard

Total Bookings
1

Completed
1

Total Spent
₹3500

Payments Done
1

Recent Bookings

Service	Price	Payment
Laptop Repair	₹3500	✔ Paid

Book New Service

Choose a category to continue

Home Cleaning

Electrician

Plumber

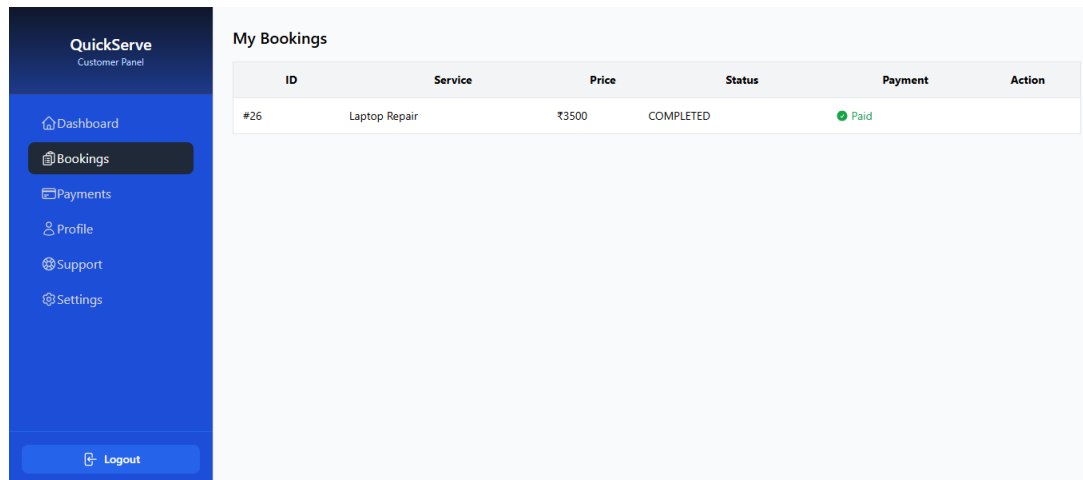
AC Repair

Appliance Service

*Booking flow handled by service module

Figure 9: Customer Dashboard

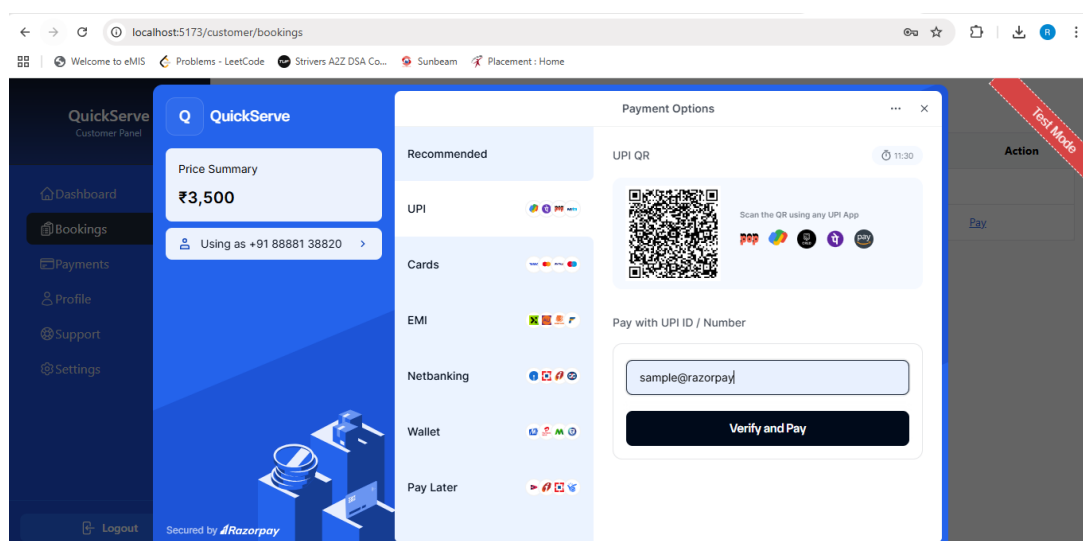
Booking Page



ID	Service	Price	Status	Payment	Action
#26	Laptop Repair	₹3500	COMPLETED	Paid	

Figure 10: Booking Page

Payment Page



QuickServe Customer Panel

Price Summary

₹3,500

Using as +91 88881 38820

Payment Options

Recommended

- UPI
- Cards
- EMI
- Netbanking
- Wallet
- Pay Later

UPI QR

Scan the QR using any UPI App

Pay with UPI ID / Number

sample@razorpay

Verify and Pay

Test Mode

Figure 11: Payment Page

Payment success Page

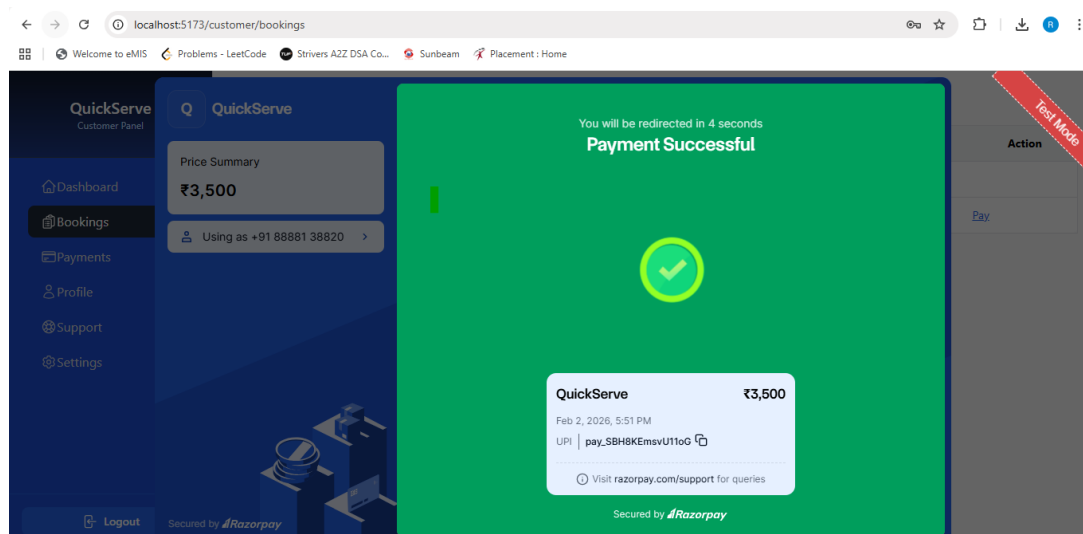


Figure 12: Payment Page

Profile Page

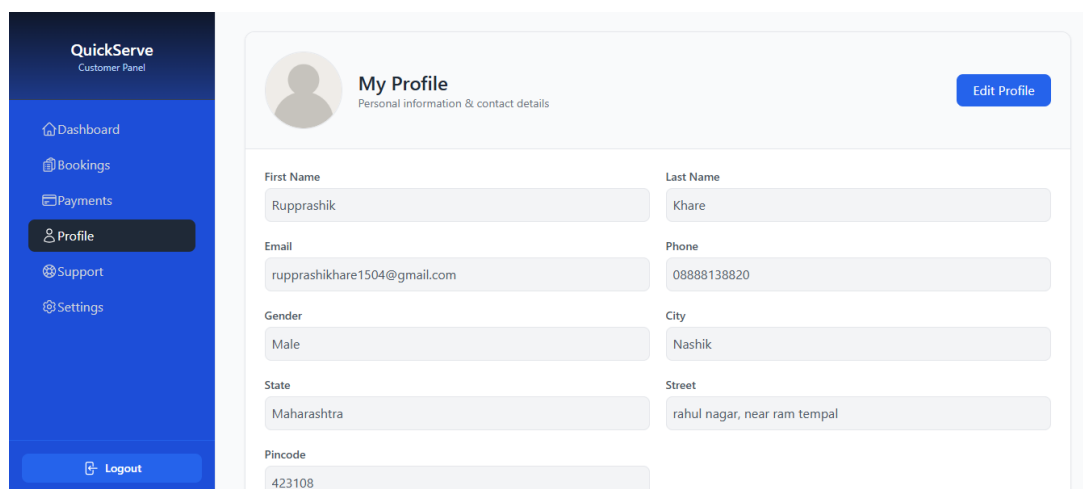


Figure 13: Profile Page

Service Provider Dashboard

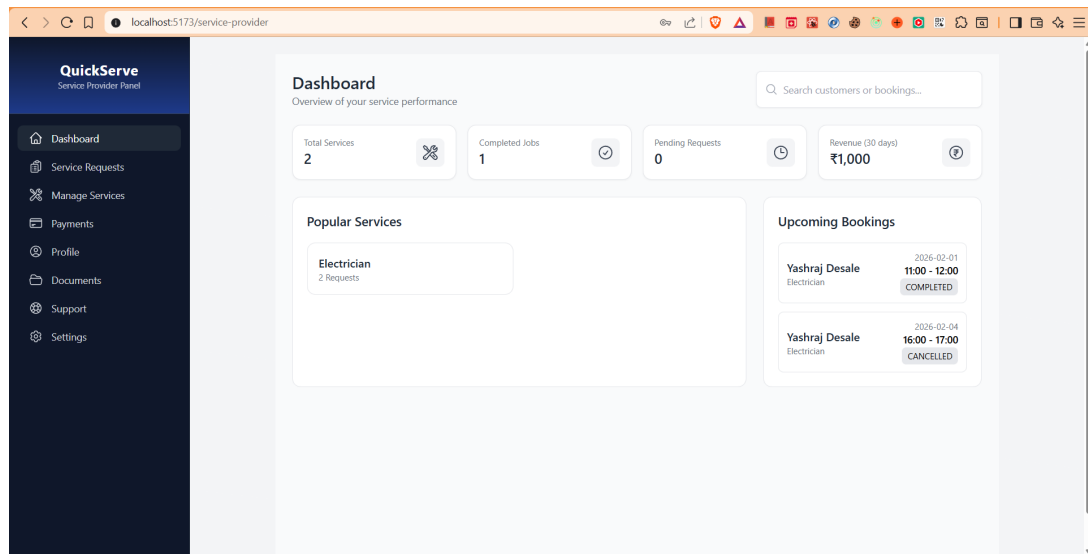


Figure 14: Service Provider Dashboard

Service Provider Booking Requests

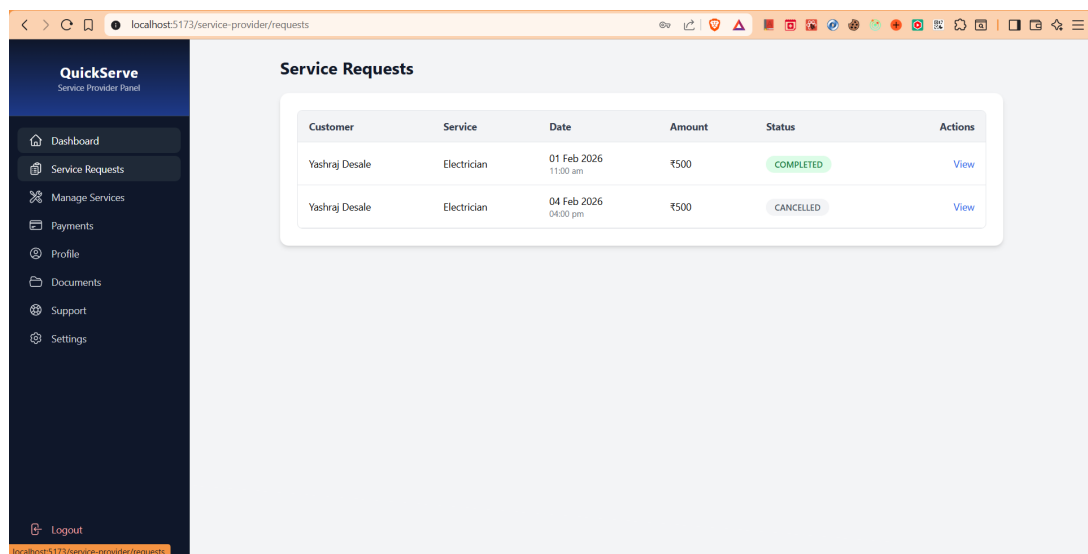


Figure 15: Service Provider Booking Requests

Service Provider Manage Services

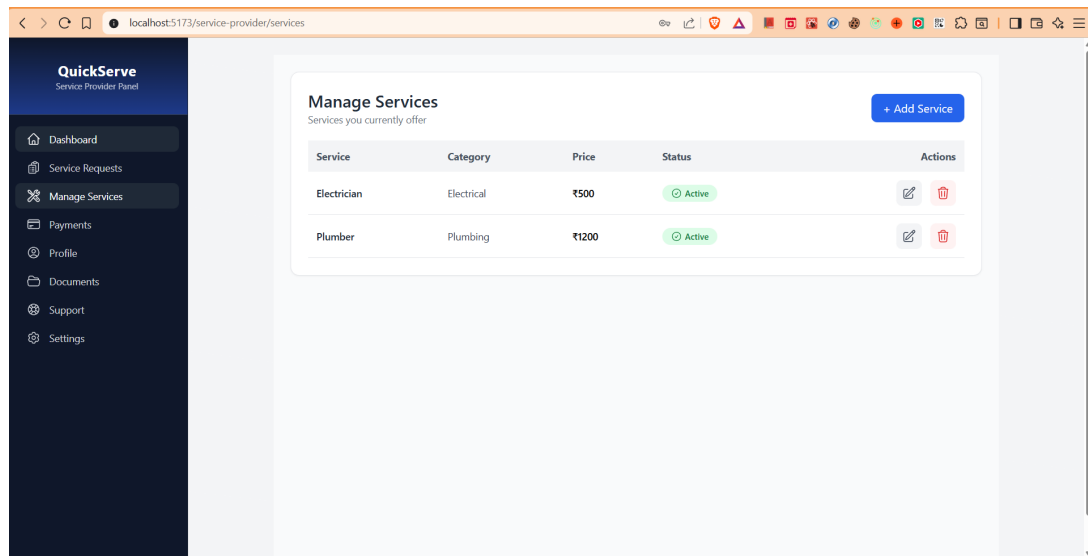


Figure 16: Manage Services

Service Provider Earnings Page

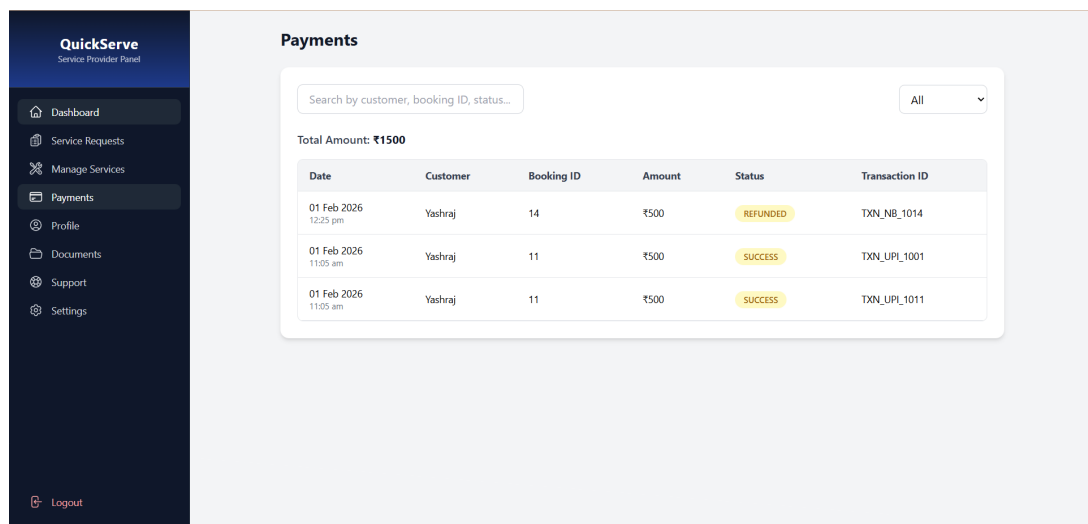


Figure 17: Service Provider Earnings

Service Provider Profile Page

The screenshot shows the 'My Profile' page for a service provider. On the left is a dark sidebar with the 'QuickServe Service Provider Panel' header and a menu including Dashboard, Service Requests, Manage Services, Payments, Profile (selected), Documents, Support, and Settings. At the bottom of the sidebar is a 'Logout' button. The main content area is titled 'My Profile' with the subtitle 'Update your personal details'. It features a circular profile picture placeholder with the letter 'r' and an 'Upload Avatar' button. To the right of the avatar are form fields for 'First Name' (value: rups), 'Last Name' (value: khare), 'Phone' (value: 9587885877), 'Date of Birth' (placeholder: dd-mm-yyyy), 'City', 'State', 'Pincode', and 'Gender' (a dropdown menu with 'Select' as the current option). At the bottom right of the form are 'Reset' and 'Save Changes' buttons.

Figure 18: Service Provider Profile

Admin Dashboard

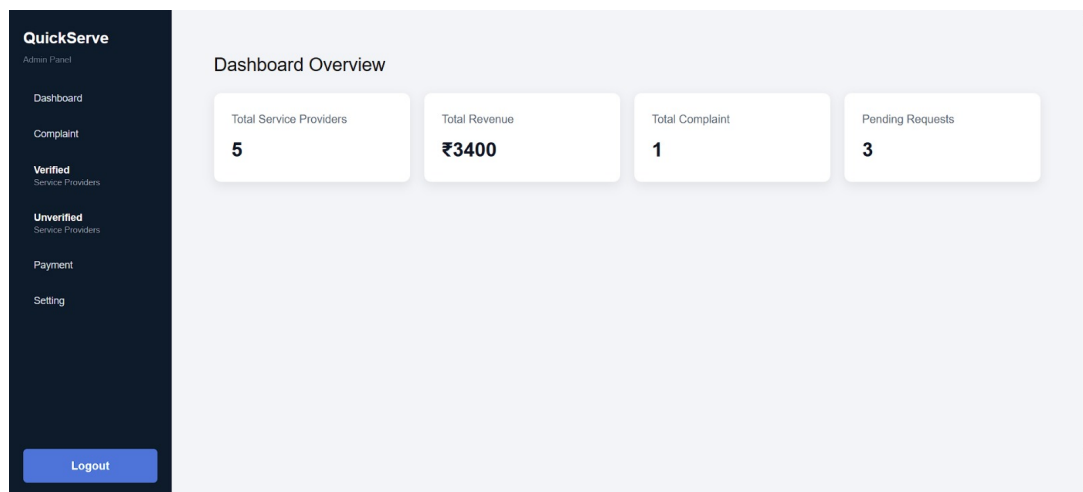


Figure 19: Admin Dashboard

User Management Page

The screenshot displays the 'Customer Complaints' section of the QuickServe Admin Panel. On the left is a dark sidebar with navigation links: Dashboard, Complaint, Verified Service Providers, Unverified Service Providers, Payment, and Setting, along with a Logout button at the bottom. The main content area has a title 'Customer Complaints' and a search bar. Below the search bar is a table with columns: ID, Name, Email, Phone, Status, and Actions. The table contains four rows of complaint data.

ID	Name	Email	Phone	Status	Actions
1	Yashraj Desale	yash@gmail.com	9307338055	RESOLVED	—
13	Yashraj Desale	yash@gmail.com	9307338055	RESOLVED	—
14	Yashraj Desale	yash@gmail.com	9307338055	OPEN	Start Reject
15	Yashraj Desale	yash@gmail.com	9307338055	IN_PROGRESS	Resolve

Figure 20: User Management

Service Provider Verification

The screenshot displays the 'Service Providers' section of the QuickServe Admin Panel. The sidebar is identical to the previous figure. The main content area has a title 'Service Providers' and a search bar. Below the search bar is a table with columns: ID, Name, Email, and Action. The table contains five rows of service provider data, each with 'View' and 'Delete' action buttons.

ID	Name	Email	Action
1	rups khare	rups@gmail.com	View Delete
2	onkar patil	onkar@gmail.com	View Delete
3	suraj shinde	s@gmail.com	View Delete
14	Krishna Yadav	krishna@gmail.com	View Delete
15	Ajay Singh	ajay@gmail.com	View Delete

Figure 21: Service Provider Verification

Manage Categories and Services

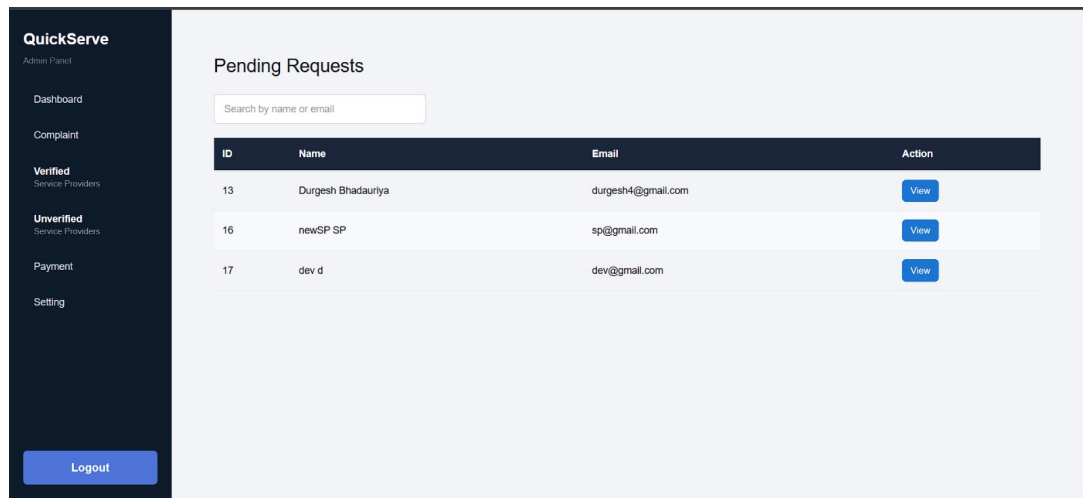


Figure 22: Manage Categories and Services

Dispute Management Page

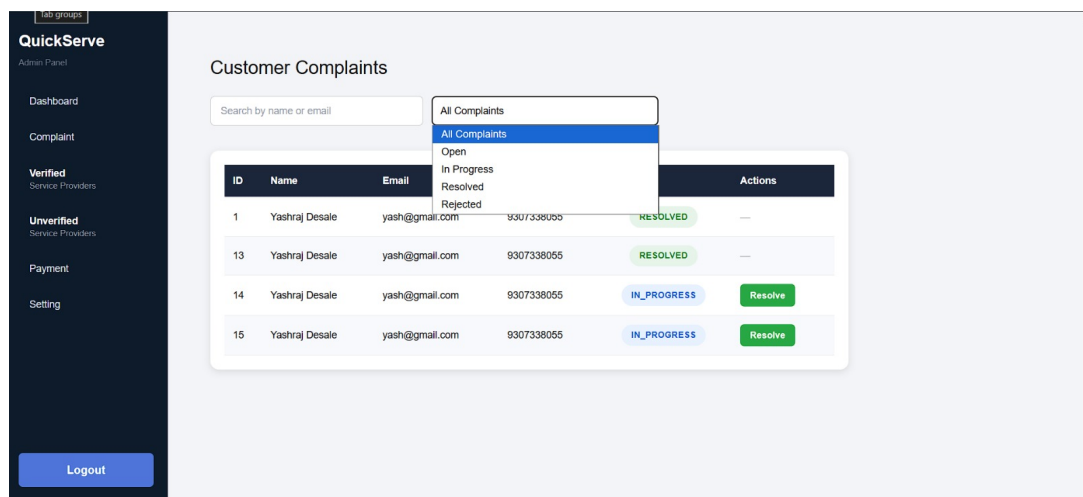


Figure 23: Dispute Management

Git log

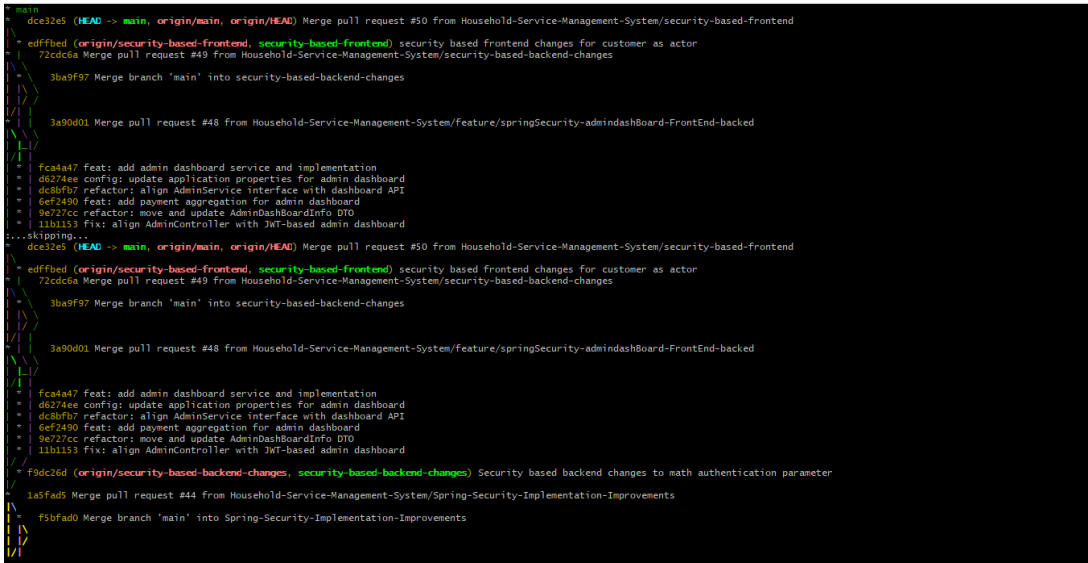


Figure 24: Git log graph

Github details

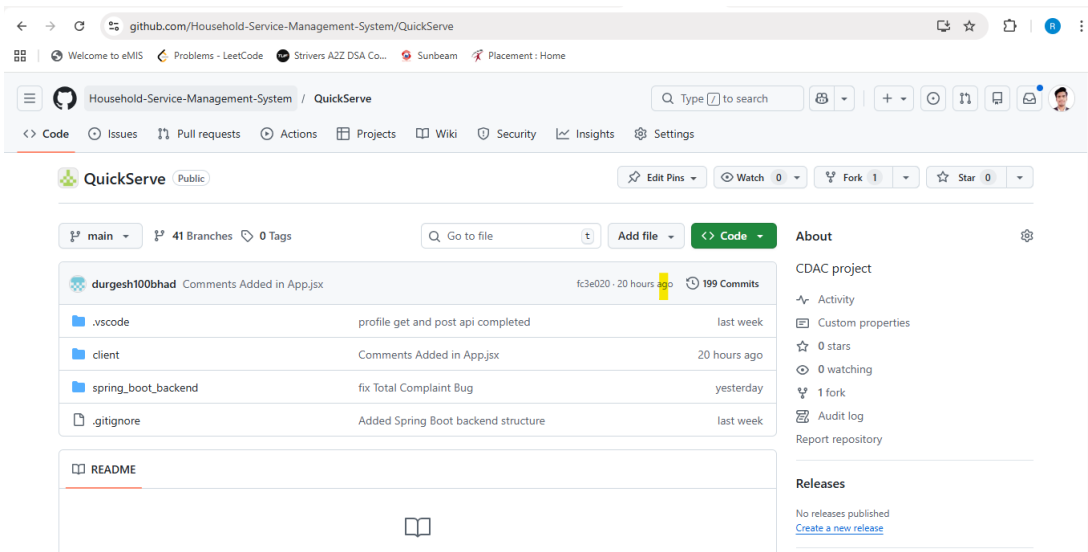


Figure 25: Github details

References

- [Spring Boot Official Documentation](#)
- [React.js Official Documentation](#)
- [Razorpay Developer Documentation](#)
- [JWT Authentication Standards](#)
- [MySQL Documentation](#)